

Dataflow Optimization within High-Level Synthesis: A Technical Review, Contextualization, and Critical Evaluation

Jaleel Ur Rehman
Electronics Engineering
Hochschule Hamm-Lippstadt

jaleel.ur-rehman@stud.hshl.de

Abstract: High-Level Synthesis (HLS) automates the translation of a behavioral description into a register-transfer-level (RTL) circuit, and the quality of the result depends heavily on how the underlying data-flow graph (DFG) is scheduled and optimized. Two established strategies expose a tradeoff: flat synthesis gives good quality but does not scale to large designs, while hierarchical synthesis is fast but misses optimizations across module boundaries. This paper reviews the hierarchical iterative-improvement technique of Lakshminarayana and Jha, which represents a design as an arbitrarily deep hierarchy of DFGs and repeatedly applies four local moves, Replace, Resynthesis, Merge, and Split, guided by timing slack computed from a global view of the hierarchy. The paper explains the scheduling and slack formalism behind the algorithm, places the work within the broader landscape of HLS scheduling and power-optimization research, and compares it with flat iterative improvement, transformational, and ILP-based alternatives. The assumptions, strengths, and weaknesses of the technique are then critically evaluated, in particular its restriction to data-flow-dominated behavior, the NP-hardness of module merging, and the missing convergence proof, and the discussion covers where the approach is and is not applicable in embedded and real-time system design.

Index Terms: high-level synthesis, data-flow graph, hierarchical synthesis, iterative improvement, power optimization, area optimization, embedded systems, ASIC design

I. INTRODUCTION

Designing large application-specific integrated circuits (ASICs) directly at the register-transfer level (RTL) is no longer practical, because the size and complexity of modern designs has grown far beyond what a manual flow can handle in reasonable time. High-Level Synthesis (HLS) addresses this problem by automatically converting a behavioral description of a circuit into an RTL implementation. The quality of the generated circuit, in terms of area, power, and timing, is determined largely by how well the data-flow graph (DFG) underlying the behavior is scheduled, bound to hardware units, and optimized [1].

Two synthesis strategies are commonly used, and each one has a clear weakness. Flat synthesis considers the entire design as a single large DFG. It typically produces good quality results, but the runtime grows too fast to be usable on large, hierarchically structured designs. Hierarchical synthesis instead optimizes each block of the design hierarchy on its own. This makes it fast, but it also means that optimization opportunities which cross the boundary between two blocks, for example sharing one functional unit between two sub-blocks, are simply never found. In short, no earlier method managed to combine good solution quality with short synthesis time for large hierarchical designs [1].

The paper reviewed in this work, by Lakshminarayana and Jha [1], was proposed to close exactly this gap. The design is represented as an arbitrarily deep hierarchy of DFGs, and an iterative improvement procedure is run on top of it that uses a global view of timing slack across the whole hierarchy, pushing that slack down to guide local module selection and resynthesis decisions. The reported results are notable: up to $6.7\times$ power reduction compared to an area-optimized baseline, and roughly $4\times$ faster synthesis than the flat approach on the tested benchmarks. Both numbers matter directly for embedded hardware such as DSP and image-processing chips, where power budgets and design-cycle time are both tightly constrained.

The rest of the paper is organized as follows. Section II explains how the hierarchical iterative-improvement algorithm actually works. Section III places the technique within the wider HLS research landscape and compares it against related scheduling and power-optimization methods. Section IV critically evaluates its assumptions, strengths, and weaknesses. Section V discusses where the technique fits and where it does not. Section VI concludes.

II. TECHNICAL DESCRIPTION

A. Hierarchical Data-Flow Representation

The design is modeled as a directed graph $G = (V, E)$, where the nodes in V are arithmetic operations and a directed edge $(u, v) \in E$ encodes a data dependency: operation v cannot begin until operation u has finished. A hierarchical

node is a node whose own internal behavior is itself described by a sub-DFG and realized by an RTL module. This module can be something simple, such as an adder, or something complex and reusable, such as an FFT block or a digital filter [1]. Because this structure is recursive, the hierarchy can be arbitrarily deep, which reflects how real designs are typically built from smaller, reusable sub-blocks rather than as one flat description.

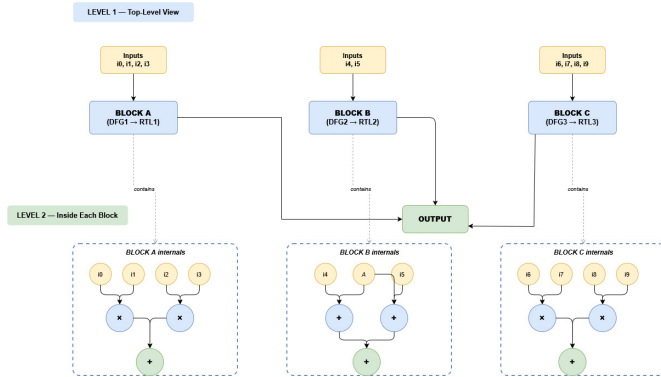


Fig. 1. Hierarchical view of a design: top-level blocks are connected to form the overall circuit, while each block internally contains its own sub-DFG realized in RTL (adapted from the seminar presentation, after [1]).

B. Scheduling and Slack

Given a fixed resource binding, the start time of an operation v is

$$t(v) = \max_{(u,v) \in E} [t(u) + d(u)] \quad (1)$$

where $d(u)$ is the delay of operation u measured in clock cycles. Ordering constraints that arise from sharing one functional unit between several operations are modeled as extra edges in the graph, which turns scheduling into a longest-path computation with a cost of $O(|V| + |E|)$ [1]. Slack is defined as the number of spare clock cycles an operation has before its deadline would be missed. When the slack of an operation is zero, only a fast, high-power unit can be used; when the slack is positive, a slower, lower-power unit can be substituted instead without violating the deadline. This slack is the quantity that drives most of the optimization decisions described below.

C. Iterative Improvement Moves

Starting from an initial solution, the algorithm repeatedly tries one of four moves until no move improves the solution any further. This is done for every candidate combination of supply voltage V_{dd} and clock period, since both directly influence power and timing:

- **Replace:** substitute an RTL module with a functionally equivalent one that fits the current slack and objective better.
- **Resynthesis:** rebuild the internal implementation of a module under a relaxed timing constraint, made possible by available slack.

- **Merge:** combine two separate RTL modules, even structurally non-isomorphic ones, into a single shared module to reduce area.
- **Split:** break a previously shared module back apart into two, which gives the search a way to escape local minima.

The central idea behind the algorithm is that a global view of the whole hierarchy exposes slack that would otherwise stay hidden if each block were optimized on its own. This slack is propagated downward and used to drive fine-grained resynthesis decisions at the lower levels of the hierarchy [1]. At the outer level, the algorithm simultaneously optimizes for area or power under a fixed throughput constraint while also choosing the supply voltage and clock period.

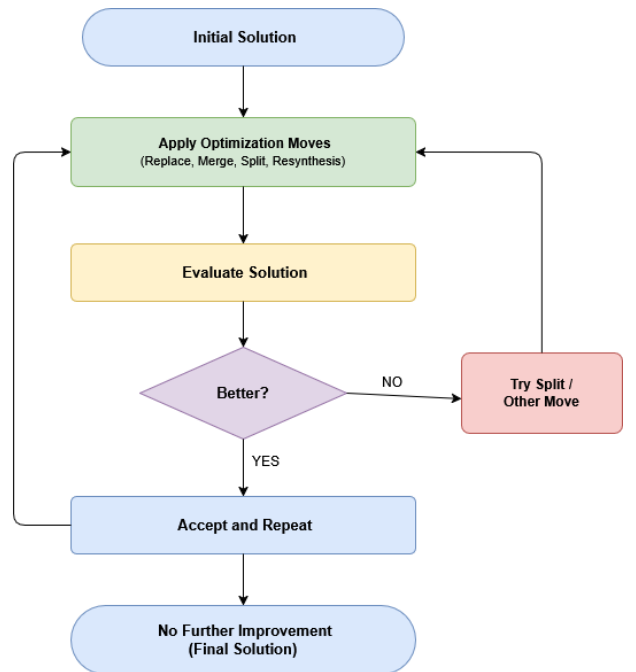


Fig. 2. Iterative improvement loop: starting from an initial solution, moves are applied and evaluated; improving solutions are accepted and repeated, while non-improving ones trigger a Split or alternative move, until no further improvement is found (adapted from the seminar presentation, after [1]).

III. SCIENTIFIC CONTEXTUALIZATION

A. Position in the HLS Landscape

Research on HLS can be roughly divided into two lines: scheduling algorithms, which decide when operations run, and resource optimization, which decides how hardware units are allocated and shared to reduce area or power. The technique reviewed here sits at the intersection of both lines, since it performs scheduling, module selection, resource sharing, and area/power optimization all at once, inside a single hierarchical iterative-improvement loop. Most earlier work treated these problems separately, or handled them jointly only for flat, non-hierarchical designs. The contribution of [1] is to combine all of them for hierarchical DFGs.

B. Related and Competing Approaches

List-based and force-directed scheduling assign operations to cycles by priority, or by balancing resource usage across cycles [4]. Both are fast, but neither directly optimizes power. ILP-based scheduling can find an optimal solution, but its runtime grows exponentially and becomes impractical once the design is not small. Simulated annealing, which the authors of [1] used as inspiration for their variable-depth improvement strategy, is able to escape local minima but tends to converge slowly. Heuristic hardware allocation algorithms proposed for datapath synthesis [5] address a related resource-binding problem, but do not target hierarchical designs or joint power and area optimization.

On the power-optimization side, transformational approaches apply algebraic rewrites to a DFG, such as retiming or exploiting associativity and commutativity, in order to reduce switching activity, but they only work on flat DFGs [3]. Operand isolation inserts gating logic so that a unit does not switch when its output is not needed; this is complementary to the reviewed technique but not something it integrates directly. Voltage scaling reduces dynamic power quadratically and is in fact used directly inside the outer V_{dd} /clock search loop of the reviewed method. The closest prior work is flat iterative improvement [2], which already solves allocation, binding, module selection, scheduling, and V_{dd} /clock selection jointly, but only for flat DFGs. The paper reviewed here extends that framework to arbitrarily deep hierarchies and additionally introduces merging of non-isomorphic modules.

C. Key Tradeoffs

Table I summarizes the main design tradeoffs that the technique has to navigate.

TABLE I
MAIN DESIGN TRADEOFFS IN DATAFLOW OPTIMIZATION FOR HLS

Tradeoff	Favours speed / simplicity	Favours quality / guarantees
Synthesis strategy	Hierarchical: fast, misses cross-block optimizations	Flat: high quality, does not scale
Area vs. power	Merge reduces area	Sharing raises switching activity, more power
V_{dd} reduction	Power scales quadratically with V_{dd}	Slower gates tighten timing margin
Optimality vs. runtime	Heuristic moves are fast, scale to real designs	ILP is optimal but exponential
Module merging	Fewer functional units, smaller area	NP-hard matching, added multiplexers

D. Consequences for Embedded and Real-Time Systems

The sampling period is enforced as a hard constraint throughout the whole synthesis process, and every single move is checked against it before being accepted, which makes the method suitable for real-time DSP systems where a missed

deadline is not acceptable. Combining V_{dd} scaling with module selection and resource sharing achieves the reported $6.7\times$ power reduction, which is directly relevant for battery-powered embedded devices such as wearables or sensor nodes. Module merging reduces the number of functional units and therefore reduces chip area, with a secondary, not directly optimized, benefit of lower interconnect power. The schedule produced by the synthesis is fixed and therefore deterministic, which is valuable for safety-critical embedded systems that require worst-case execution time (WCET) analysis. The method targets ASICs; applying it to FPGAs would require adapting the resource and area model to an LUT-based one [1].

Hierarchical, power-optimized HLS was still a fairly new problem area at the time this work was published, and non-isomorphic RTL module merging in particular had no direct predecessor, which is why directly comparable prior work is somewhat sparse. Control-flow-intensive HLS is a neighboring but technically different problem class, and the seed paper explicitly leaves it as future work rather than attempting to cover it.

IV. CRITICAL EVALUATION

A. Strengths

Speed and quality together. The technique is reported to be roughly $4\times$ faster than flat synthesis while still producing circuits of comparable quality, a combination that no earlier hierarchical method achieved at the same time [1].

Non-isomorphic module sharing. The Merge move allows two structurally different DFGs to share a single RTL module. Earlier methods required separate modules in this case, so this removes a significant source of area overhead.

Global slack propagation. Using the global view of the hierarchy to find slack, and then pushing it down to drive fine-grained resynthesis, exposes power savings that simply cannot be found when a block is optimized in isolation.

Validated at layout level. The $6.7\times$ power reduction is measured after actual placement and routing rather than only estimated at a higher level of abstraction, which is a stronger form of validation than most HLS papers provide.

B. Limitations and Risks

Data-flow only. Designs with meaningful control flow, such as conditional branches or variable-bound loops, are explicitly outside the scope of the method, even though most realistic embedded applications combine computation with control logic.

NP-hard module merging. The RTL module-merging problem is at least as hard as graph isomorphism. The heuristic used to solve it may miss the optimal merging, and how large the resulting quality gap is remains unknown for larger modules.

No convergence proof. A cycle between the Merge and Split moves is theoretically possible, and the paper gives no formal proof that the algorithm always terminates.

Approximate power model. The switched-capacitance model used to estimate power ignores glitch power and

detailed interconnect effects, so some of the optimization decisions made during synthesis could turn out to be wrong once measured accurately at the layout level.

C. Assumptions That May Be Unrealistic

Fixed sampling period. The throughput constraint is fixed at design time. In adaptive systems whose deadlines change during runtime, this is a real limitation, since the method has no built-in mechanism to handle variable deadlines.

Typical input statistics. The power model is built on typical input traces. If the actual worst-case inputs produce very different switching activity, the reported power savings could turn out to be optimistic in practice.

User-declared equivalence. The Replace move partly depends on the user declaring which DFGs are functionally equivalent to each other. In practice this kind of information is rarely available in a clean, structured form.

V. APPLICATION DISCUSSION

The technique, and approaches derived from it, are well suited for systems with the following properties:

- **Data-dominated ASIC designs with fixed throughput requirements:** DSP chips (FIR, IIR, DCT, FFT) and image or video processing ASICs fit the data-flow-only assumption of the method directly [1].
- **Battery-powered and resource-constrained embedded devices:** IoT sensor nodes and similar devices benefit directly from the reported power reduction, since every milliwatt saved extends operating time.

On the other hand, the method is not a good fit for control-flow intensive embedded firmware, since such designs fall outside its data-flow-only scope. It is also not directly usable for FPGA targets without adapting the resource model, is unsuitable for systems whose timing requirements are not known statically, and cannot be applied where an optimality guarantee is formally required, since the underlying heuristics offer no such guarantee [1].

VI. CONCLUSION

The hierarchical iterative-improvement technique proposed by Lakshminarayana and Jha resolves a tradeoff that had limited earlier HLS methods for a long time: the choice between synthesis speed and solution quality. By taking a global view of a hierarchical DFG and propagating slack downward to drive four local moves, Replace, Resynthesis, Merge, and Split, the method reaches both good quality and short runtime at the same time. The reported results show substantial power reduction, validated at the layout level, together with a clearly shorter synthesis time than flat approaches on the tested benchmarks [1].

At the same time, the method has real limitations. It only works for data-flow-dominated behavior, the module-merging step is NP-hard and only solved heuristically, and no formal proof is given that the algorithm always converges. For embedded systems with fixed, data-dominated workloads and tight power or area budgets, the technique is still a relevant

reference point today. Extending it to control-flow-intensive behavior and to FPGA-style resource models remain the most promising directions for future work.

REFERENCES

- [1] G. Lakshminarayana and N. K. Jha, "High-Level Synthesis of Power-Optimized and Area-Optimized Circuits from Hierarchical Data-Flow Intensive Behaviors," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 18, no. 2–3, pp. 265–281, 1999.
- [2] A. Raghunathan and N. K. Jha, "An Iterative Improvement Algorithm for Low Power Datapath Synthesis," in *Proc. Int. Conf. Comput.-Aided Des.*, Nov. 1995, pp. 597–602.
- [3] A. P. Chandrakasan, M. Potkonjak, R. Mehra, J. Rabaey, and R. W. Brodersen, "Optimizing Power Using Transformations," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 14, pp. 12–51, Jan. 1995.
- [4] P. G. Paulin and J. P. Knight, "Force-Directed Scheduling for the Behavioral Synthesis of ASICs," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 8, no. 6, pp. 661–679, June 1989.
- [5] S. Devadas and A. R. Newton, "Algorithms for Hardware Allocation in Data Path Synthesis," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 8, pp. 768–781, July 1989.